EDA

2026-04-17

Packages

```r
# install.packages("f1dataR")
# install.packages("dplyr")
```

```r
library(f1dataR)
library(dplyr)
library(tidyverse)
```

```r
# Load the pre-compiled dataset containing all seasons (2019-2024)
# full_data <- map_dfr(2019:2024, function(year) {
#   schedule <- load_schedule(season = year)
#   map_dfr(seq_len(nrow(schedule)), function(round) {
#     tryCatch(
#       load_results(season = year, round = round) %>%
#         mutate(
#           season = year,
#           round = round,
#           race_name = schedule$race_name[round],
#           circuit = schedule$circuit_id[round],
#           top_speed_kph = as.numeric(top_speed_kph),
#           fastest_rank = as.integer(fastest_rank)
#         ),
#       error = function(e) {
#         message("Skipping season ", year, " round ", round, ": ", e$message)
#         NULL
#       }
#     )
#   })
# })

# write_csv(full_data, "full_data.csv")

full_data <- read.csv("full_data.csv")

head(full_data, 8)
```

```
##         driver_id constructor_id points position grid laps   status         gap
## 1          bottas       mercedes     26        1    2   58 Finished 1:25:27.325
## 2        hamilton       mercedes     18        2    1   58 Finished     +20.886
## 3  max_verstappen       red_bull     15        3    4   58 Finished     +22.520
## 4          vettel        ferrari     12        4    3   58 Finished     +57.109
## 5         leclerc        ferrari     10        5    5   58 Finished     +58.203
## 6 kevin_magnussen           haas      8        6    7   58 Finished   +1:27.156
## 7        hamilton       mercedes     25        1    3   57 Finished 1:34:21.295
```

```
## 8          bottas       mercedes     18        2    4   57 Finished      +2.980
##   fastest_rank  fastest top_speed_kph time_sec season round
## 1            1 1:25.580       223.075   85.580   2019     1
## 2            2 1:26.057       221.839   86.057   2019     1
## 3            3 1:26.256       221.327   86.256   2019     1
## 4            8 1:27.954       217.054   87.954   2019     1
## 5            4 1:26.926       219.621   86.926   2019     1
## 6            9 1:28.182       216.493   88.182   2019     1
## 7            2 1:33.528       208.314   93.528   2019     2
## 8            3 1:34.209       206.808   94.209   2019     2
##               race_name     circuit gap_seconds
## 1 Australian Grand Prix albert_park       0.000
## 2 Australian Grand Prix albert_park      20.886
## 3 Australian Grand Prix albert_park      22.520
## 4 Australian Grand Prix albert_park      57.109
## 5 Australian Grand Prix albert_park      58.203
## 6 Australian Grand Prix albert_park      87.156
## 7    Bahrain Grand Prix     bahrain       0.000
## 8    Bahrain Grand Prix     bahrain       2.980
```

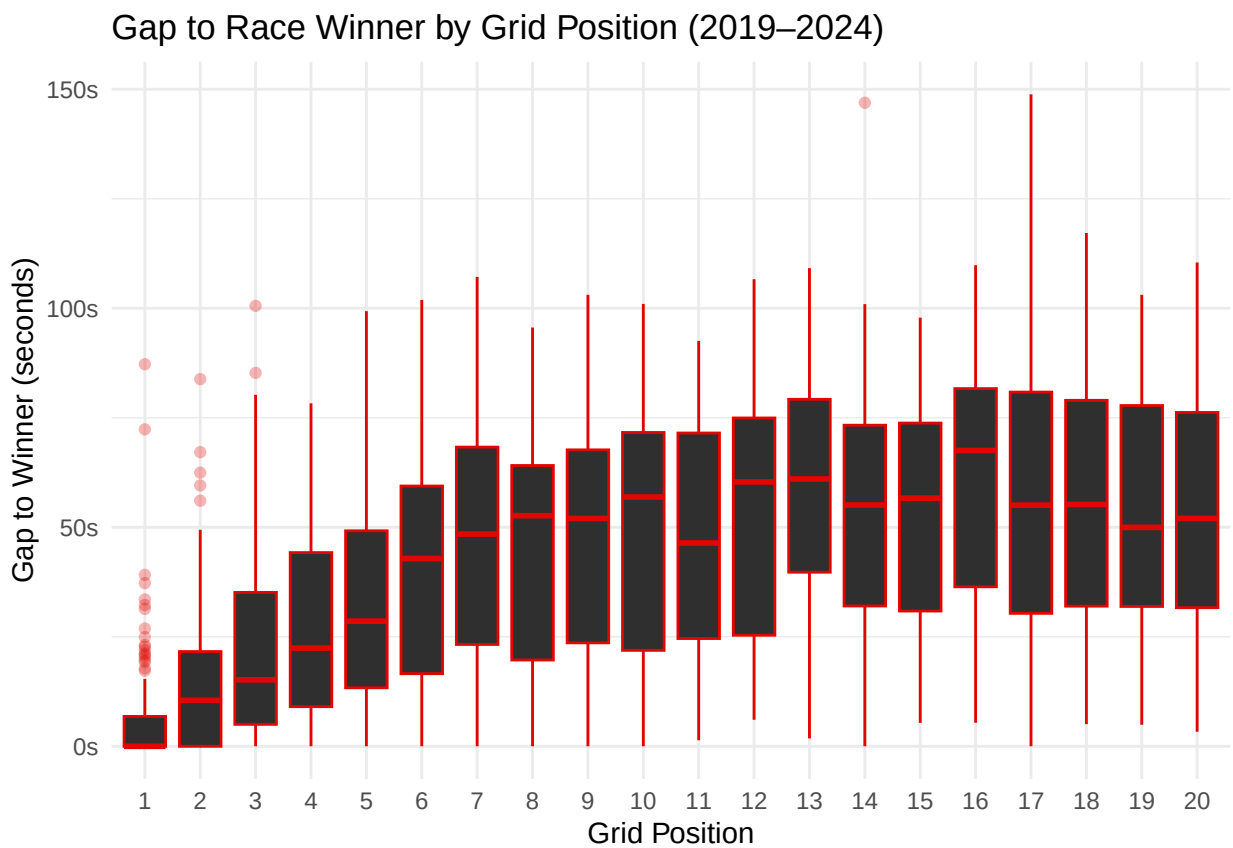
```r
# Helper function to convert gap values (strings like "+1:23.4") into numeric seconds
convert_to_seconds <- function(x) {
  if (is.numeric(x)) return(x)

  x <- as.character(x)

  sapply(x, function(val) {
    if (grepl("^[+-]?\\d+\\.?\\d*$", val)) {
      return(as.numeric(val))
    }

    if (!grepl("^[+-]", val) && grepl(":", val)) {
      return(0)
    }

    sign <- ifelse(startsWith(val, "-"), -1, 1)
    val <- gsub("^[+-]", "", val)

    parts <- strsplit(val, ":")[[1]]

    if (length(parts) == 2) {
      return(sign * (as.numeric(parts[1]) * 60 + as.numeric(parts[2])))
    } else if (length(parts) == 3) {
      return(sign * (as.numeric(parts[1]) * 3600 +
                     as.numeric(parts[2]) * 60 +
                     as.numeric(parts[3])))
    } else {
      return(NA_real_)
    }
  }, USE.NAMES = FALSE)
}
```

```r
# Clean dataset: convert gap to seconds, fix grid type, and filter valid race finishes
full_data <- full_data %>%
  mutate(
```

```r
    gap_seconds = convert_to_seconds(gap),
    grid = as.integer(grid)
  ) %>%
  filter(
    status == "Finished",
    !is.na(grid),
    grid > 0,
    !is.na(gap_seconds)
  )

head(full_data, 8)
```

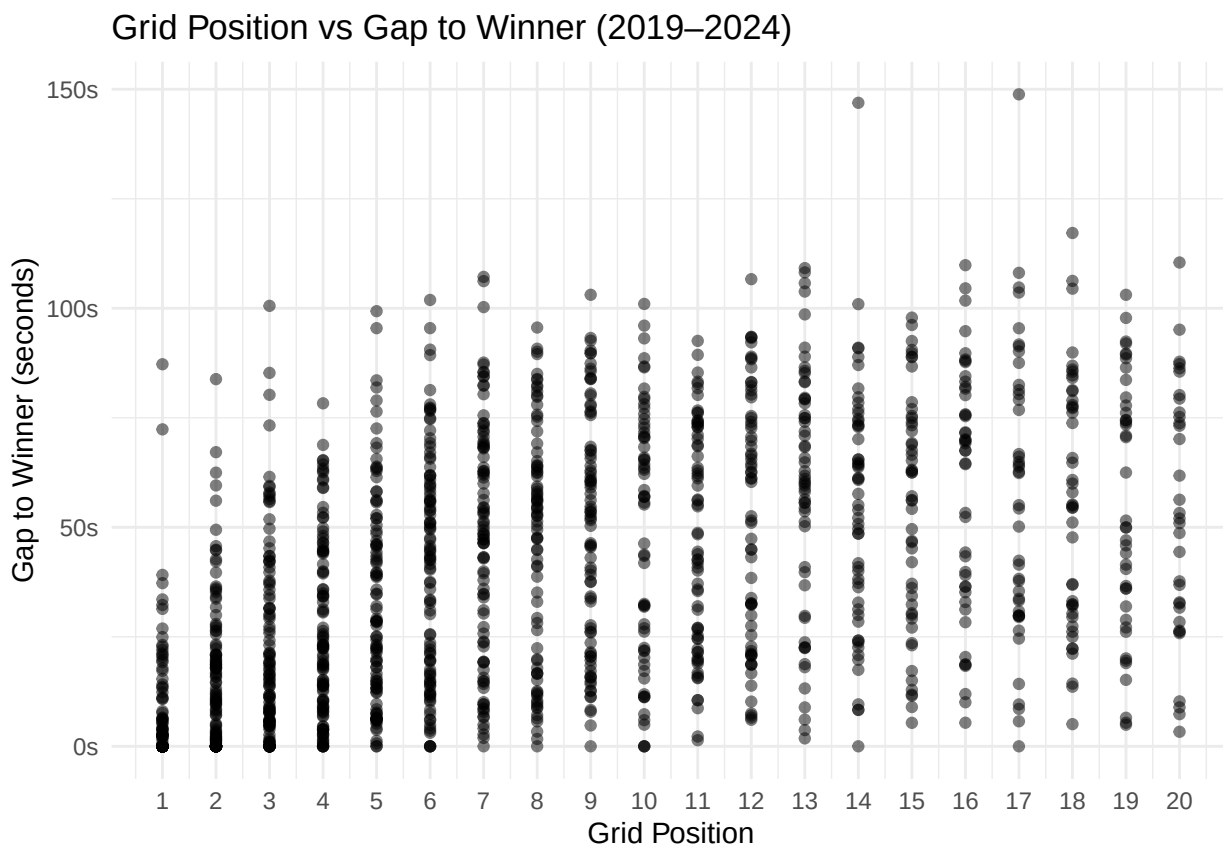
```
##         driver_id constructor_id points position grid laps   status         gap
## 1          bottas       mercedes     26        1    2   58 Finished 1:25:27.325
## 2        hamilton       mercedes     18        2    1   58 Finished     +20.886
## 3  max_verstappen       red_bull     15        3    4   58 Finished     +22.520
## 4          vettel        ferrari     12        4    3   58 Finished     +57.109
## 5         leclerc        ferrari     10        5    5   58 Finished     +58.203
## 6 kevin_magnussen           haas      8        6    7   58 Finished   +1:27.156
## 7        hamilton       mercedes     25        1    3   57 Finished 1:34:21.295
## 8          bottas       mercedes     18        2    4   57 Finished      +2.980
##   fastest_rank  fastest top_speed_kph time_sec season round
## 1            1 1:25.580       223.075   85.580   2019     1
## 2            2 1:26.057       221.839   86.057   2019     1
## 3            3 1:26.256       221.327   86.256   2019     1
## 4            8 1:27.954       217.054   87.954   2019     1
## 5            4 1:26.926       219.621   86.926   2019     1
## 6            9 1:28.182       216.493   88.182   2019     1
## 7            2 1:33.528       208.314   93.528   2019     2
## 8            3 1:34.209       206.808   94.209   2019     2
##               race_name     circuit gap_seconds
## 1 Australian Grand Prix albert_park       0.000
## 2 Australian Grand Prix albert_park      20.886
## 3 Australian Grand Prix albert_park      22.520
## 4 Australian Grand Prix albert_park      57.109
## 5 Australian Grand Prix albert_park      58.203
## 6 Australian Grand Prix albert_park      87.156
## 7    Bahrain Grand Prix     bahrain       0.000
## 8    Bahrain Grand Prix     bahrain       2.980
```

```r
# grid vs gap across all circuits
full_data %>%
  ggplot(aes(x = factor(grid), y = gap_seconds)) +
  geom_boxplot(outlier.alpha = 0.3, fill = "#2f2f2f", color = "#e10600") +
  scale_y_continuous(labels = scales::label_number(suffix = "s")) +
  labs(
    title = "Gap to Race Winner by Grid Position (2019-2024)",
    x = "Grid Position",
    y = "Gap to Winner (seconds)"
  ) +
  theme_minimal()
```



Gap to winner increases monotonically with grid position. Drivers starting from the front consistently finish closer to the winner, while higher grid positions show both larger median gaps and greater spread.

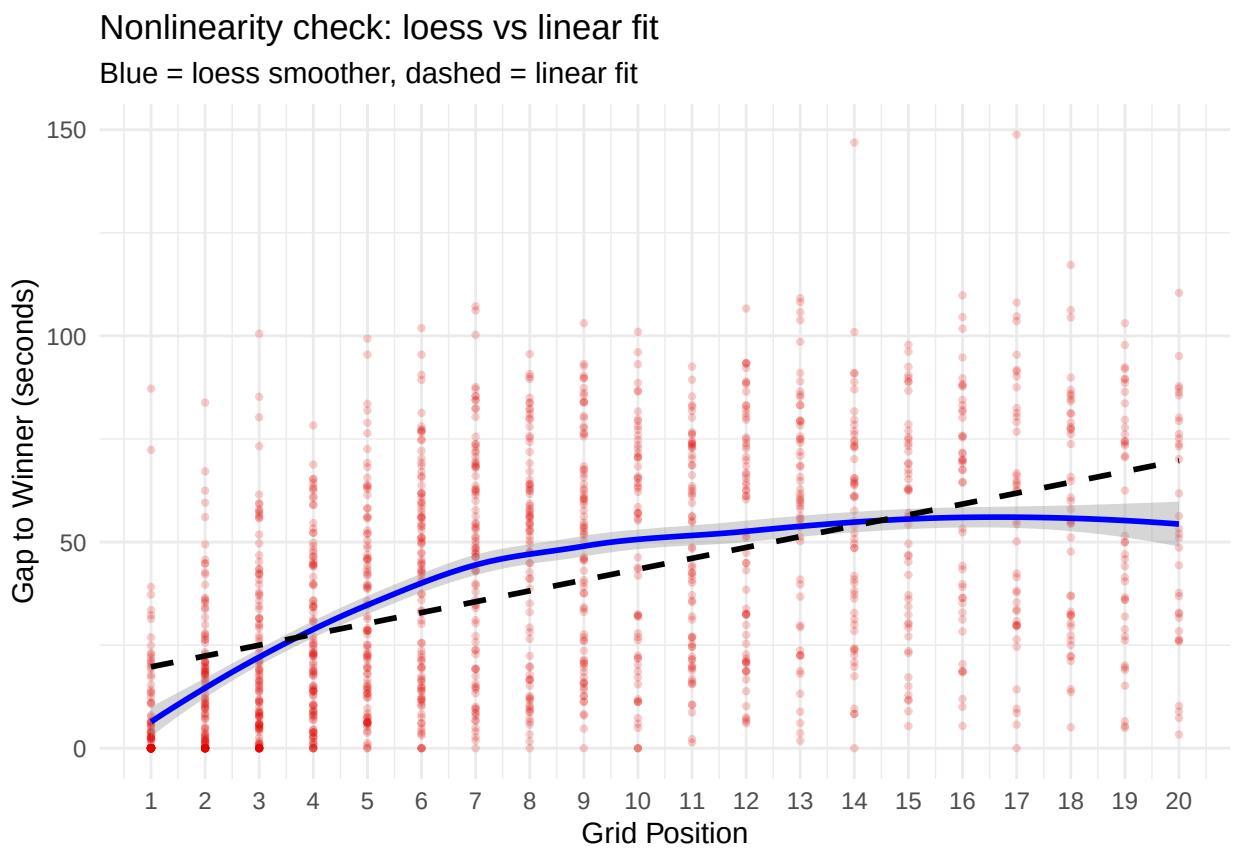
```
# grid vs gap across all circuits scatterplot
full_data %>%
  ggplot(aes(x = grid, y = gap_seconds)) +
  geom_point(alpha = 0.5, size = 1.5) +
  scale_x_continuous(breaks = 1:20) +
  scale_y_continuous(labels = scales::label_number(suffix = "s")) +
  scale_color_viridis_d(option = "turbo") +  # 20+ distinct colors
  labs(
    title = "Grid Position vs Gap to Winner (2019-2024)",
    x = "Grid Position",
    y = "Gap to Winner (seconds)"
  ) +
  theme_minimal()
```



The upward trend is clear but highly variable. Notable is the presence of gap = 0 observations across all grid positions, confirming that any starting position can produce a race winner.

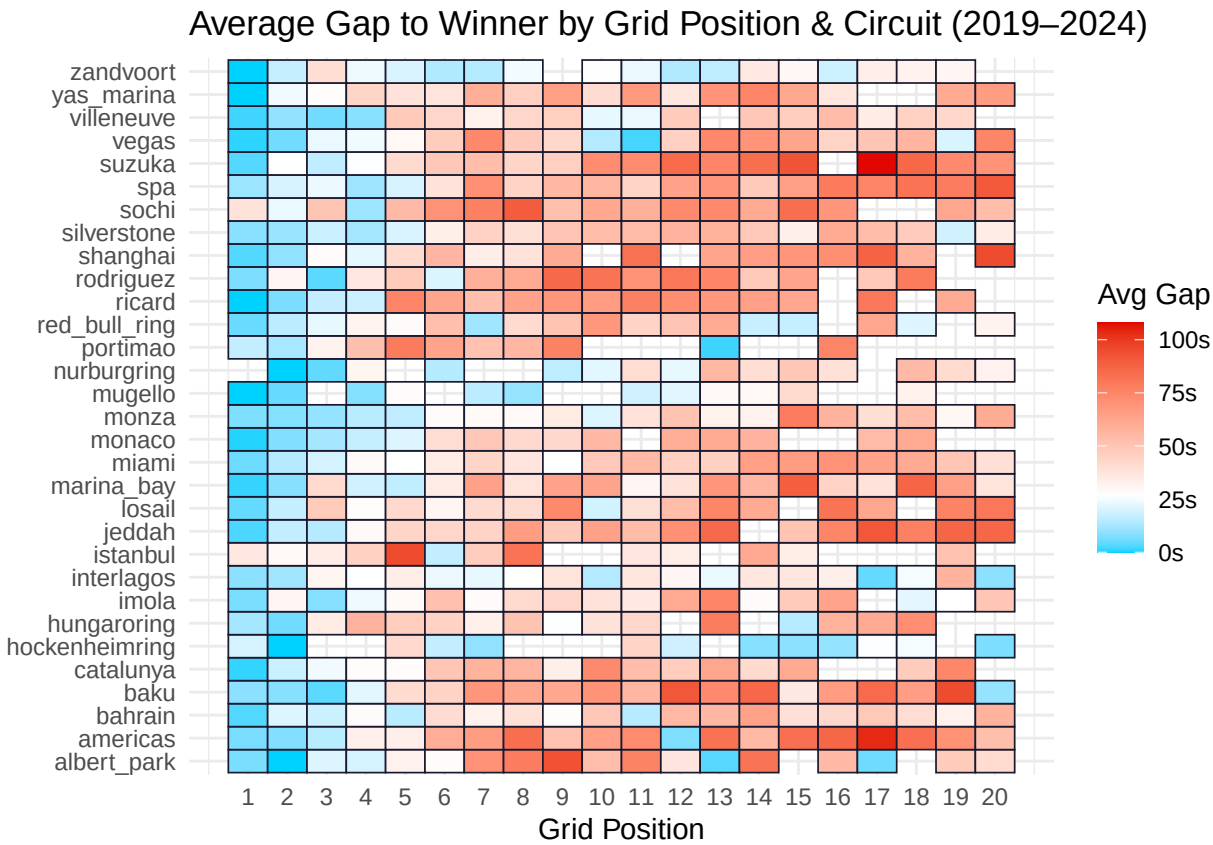
```
# Nonlinearity check: overlays loess smoother vs linear fit to assess
# whether grid position has a nonlinear effect on gap to winner

full_data %>%
  ggplot(aes(x = grid, y = gap_seconds)) +
  geom_point(alpha = 0.2, size = 0.8, color = "#e10600") +
  geom_smooth(method = "loess", color = "blue",
              se = TRUE, linewidth = 1) +
  geom_smooth(method = "lm", color = "black",
              linetype = "dashed", se = FALSE, linewidth = 1) +
  scale_x_continuous(breaks = 1:20) +
  labs(
    title = "Nonlinearity check: loess vs linear fit",
    subtitle = "Blue = loess smoother, dashed = linear fit",
    x = "Grid Position",
    y = "Gap to Winner (seconds)"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```



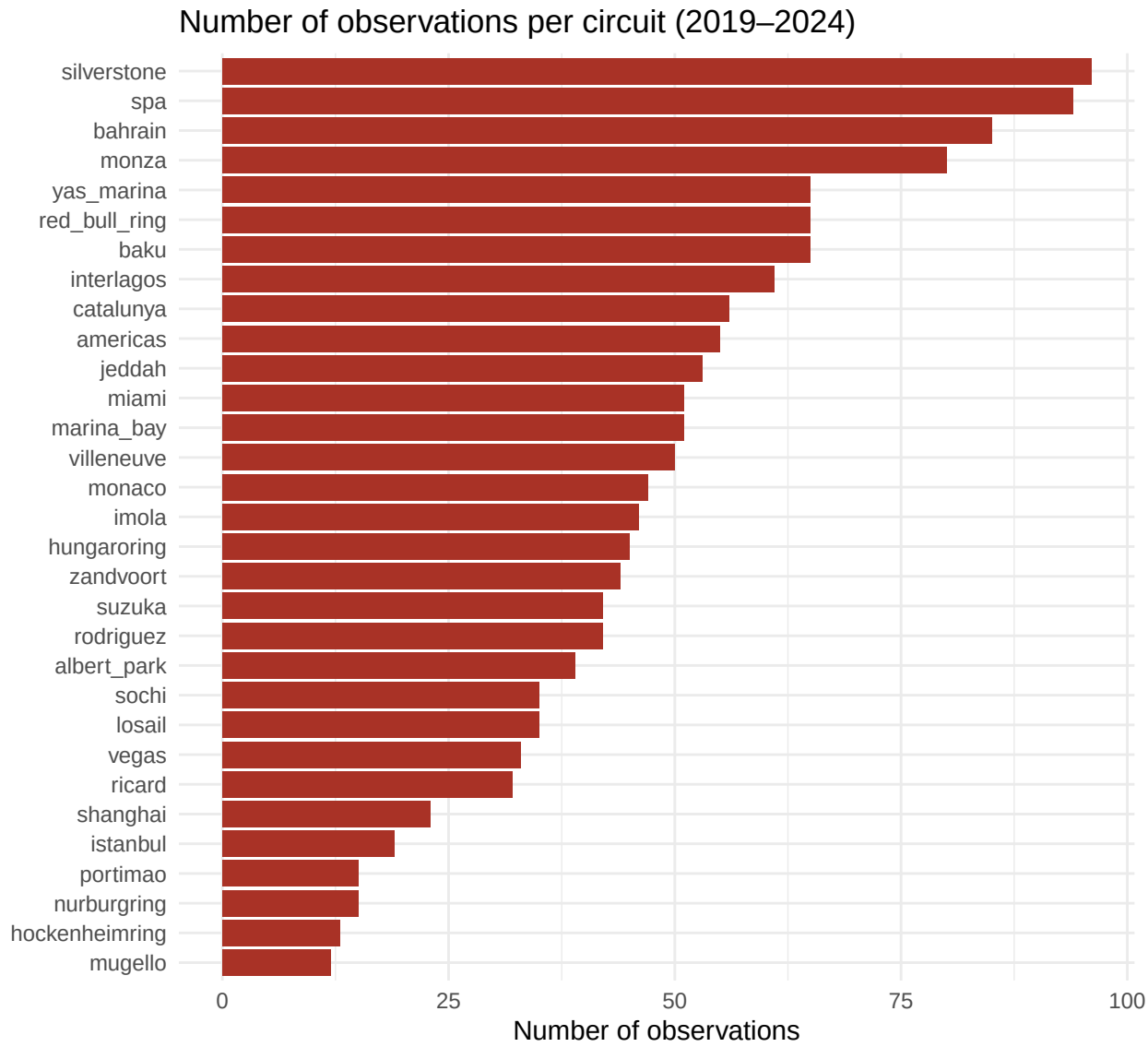
The loess smoother (blue) diverges from the linear fit (dashed black line), suggesting the relationship between grid position and gap to winner is not linear. The gap increases steeply from positions P1 to P8 but begins to flatten beyond P10, indicating diminishing returns at higher grid positions. This pattern suggests that a log transformation of the grid predictor may better capture the curvature in the data compared to a simple linear term.

```
# average gap of each grid by circuit
full_data %>%
  group_by(circuit, grid) %>%
  summarise(avg_gap = mean(gap_seconds), .groups = "drop") %>%
  ggplot(aes(x = grid, y = circuit, fill = avg_gap)) +
  geom_tile(color = "#1a1a2e", linewidth = 0.3) +
  scale_x_continuous(breaks = 1:20) +
  scale_fill_gradientn(
    colors = c("#00d2ff", "#ffffff", "#e10600"),
    values = scales::rescale(c(0, 30, 120)),  # adjust midpoint if needed
    labels = scales::label_number(suffix = "s"),
    name = "Avg Gap"
  ) +
  labs(
    title = "Average Gap to Winner by Grid Position & Circuit (2019-2024)",
    x = "Grid Position",
    y = NULL
  ) +
  theme_minimal()
```



Average gap varies across circuits, suggesting that circuit characteristics play a role in determining finishing gaps. For example, high-speed circuits like Spa and Silverstone tend to produce larger gaps, while street circuits like Monaco and Interlagos appear more compressed. This variability across circuits suggests that a single pooled model may be insufficient, motivating the use of a hierarchical model with circuit-specific effects.

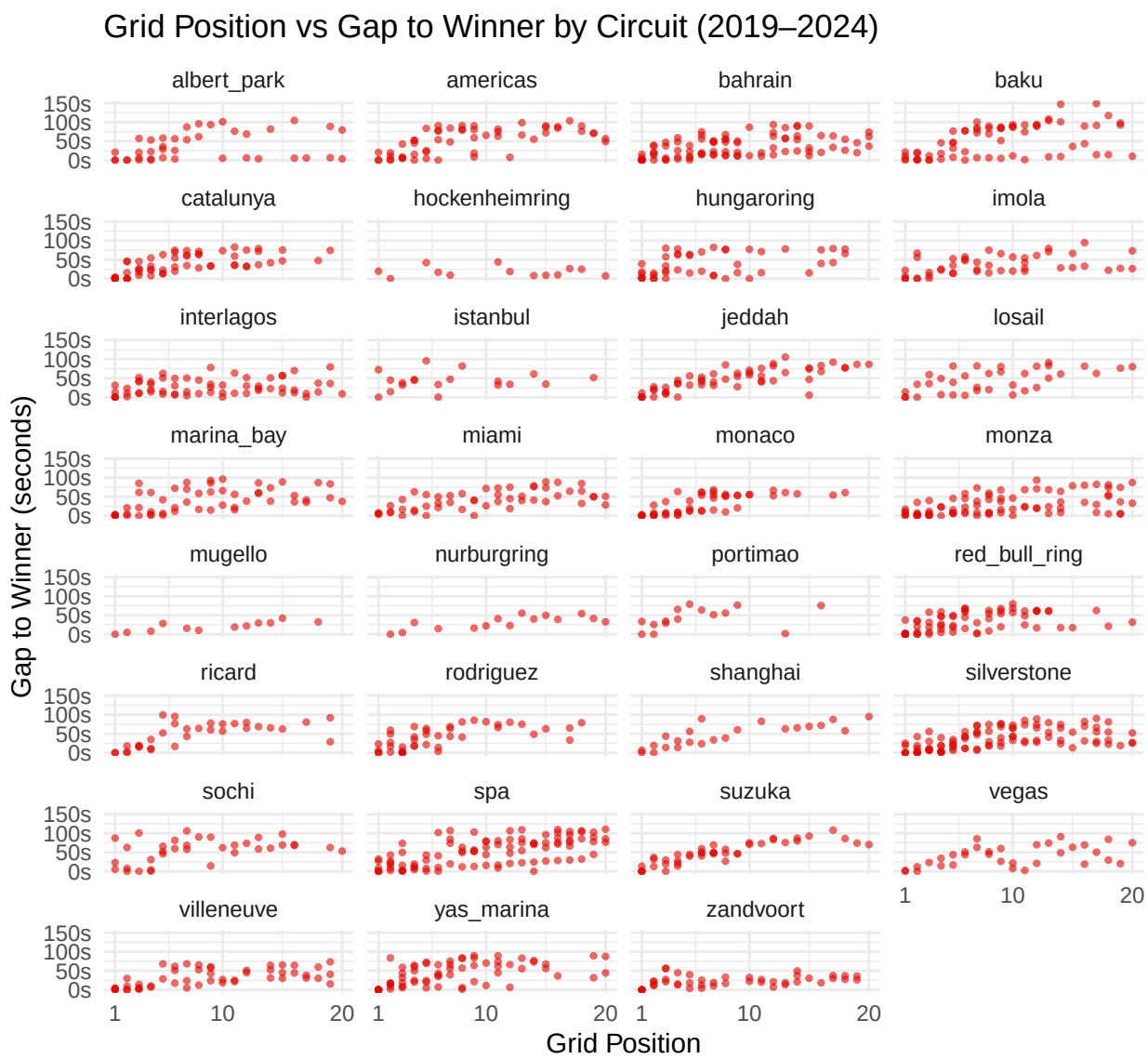
```r
# Sample size per circuit
full_data %>%
  count(circuit) %>%
  ggplot(aes(x = reorder(circuit, n), y = n)) +
  geom_col(fill = "#a93226") +
  coord_flip() +
  labs(
    title = "Number of observations per circuit (2019-2024)",
    x = NULL,
    y = "Number of observations"
  ) +
  theme_minimal()
```



The number of observations varies considerably across circuits. Established circuits such as Silverstone, Spa, and Bahrain appear across all six seasons and therefore have the most data, while newer or one-off circuits such as Mugello, Hockenheimring, and Nurburgring appear only once or twice. This imbalance means that estimates for data-sparse circuits would be unreliable if modelled independently. A hierarchical model addresses this by allowing circuits with fewer observations to borrow strength from the overall distribution, producing more stable estimates.

```
# grid vs gap for each circuit
full_data %>%
  ggplot(aes(x = grid, y = gap_seconds)) +
  geom_point(alpha = 0.6, size = 0.8, color = "#e10600") +
  facet_wrap(~ circuit, ncol = 4) +
  scale_x_continuous(breaks = c(1, 10, 20)) +
  scale_y_continuous(labels = scales::label_number(suffix = "s")) +
  labs(
    title = "Grid Position vs Gap to Winner by Circuit (2019-2024)",
    x = "Grid Position",
    y = "Gap to Winner (seconds)"
```

```
  ) +
  theme_minimal()
```



Looking at individual circuits, the strength of the grid-gap relationship appears to vary. Some circuits show a steeper positive trend while others appear relatively flat, which is consistent with the patterns seen in the heatmap. This further supports the inclusion of circuit-specific slope parameters in the hierarchical model.

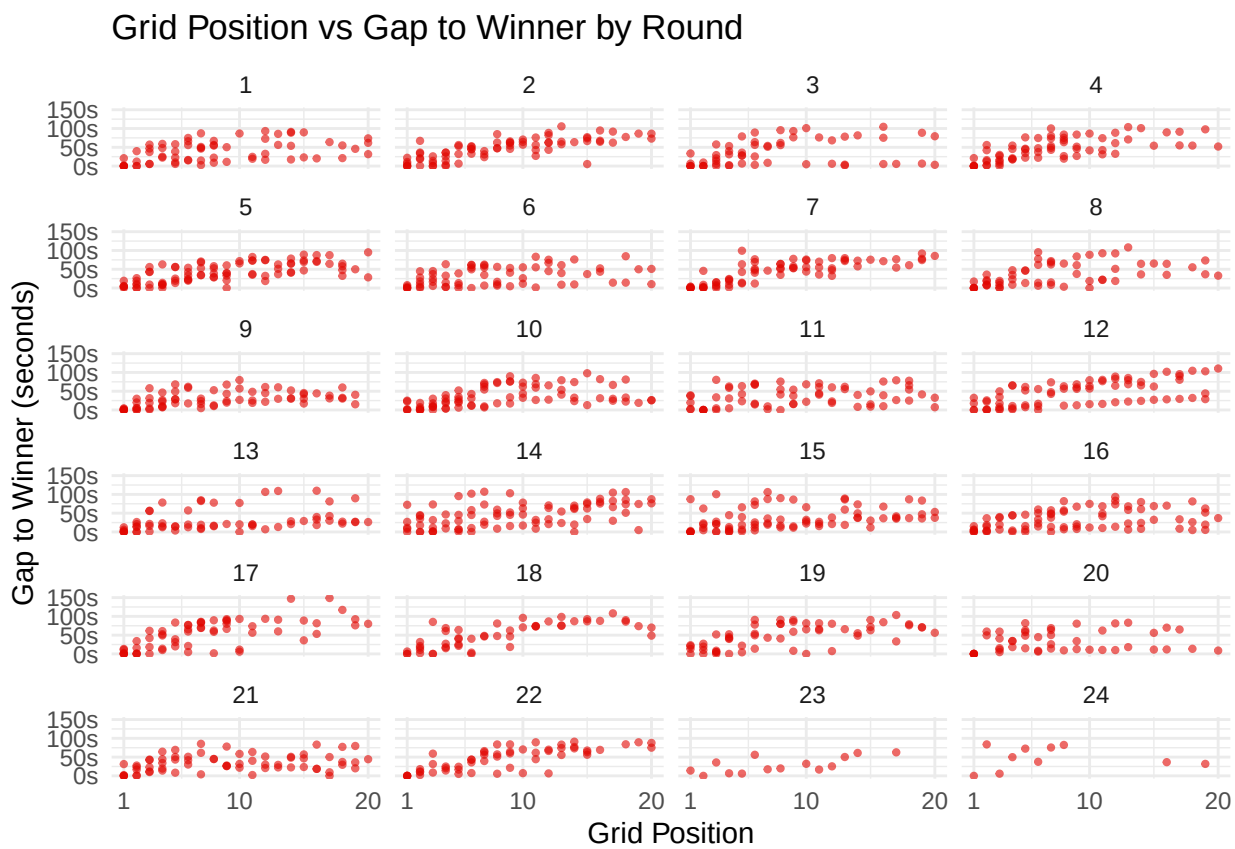
```
# grid vs gap by season
full_data %>%
  ggplot(aes(x = grid, y = gap_seconds)) +
  geom_point(alpha = 0.6, size = 0.8, color = "#e10600") +
  facet_wrap(~ season, ncol = 4) +
  scale_x_continuous(breaks = c(1, 10, 20)) +
  scale_y_continuous(labels = scales::label_number(suffix = "s")) +
  labs(
    title = "Grid Position vs Gap to Winner by Season",
    x = "Grid Position",
```

```
    y = "Gap to Winner (seconds)"
  ) +
  theme_minimal()
```

The grid-gap relationship appears broadly consistent across the 2019 to 2024 seasons, with no obvious systematic changes over time. This suggests that pooling data across seasons is a reasonable assumption for our analysis.

```
# grid vs gap by round
full_data %>%
  ggplot(aes(x = grid, y = gap_seconds)) +
```

```r
  geom_point(alpha = 0.6, size = 0.8, color = "#e10600") +
  facet_wrap(~ round, ncol = 4) +
  scale_x_continuous(breaks = c(1, 10, 20)) +
  scale_y_continuous(labels = scales::label_number(suffix = "s")) +
  labs(
    title = "Grid Position vs Gap to Winner by Round",
    x = "Grid Position",
    y = "Gap to Winner (seconds)"
  ) +
  theme_minimal()
```



No clear pattern emerges when examining gap by round number within a season. The variation observed across rounds likely reflects race-specific conditions such as weather or safety car periods rather than any structural effect of round order.

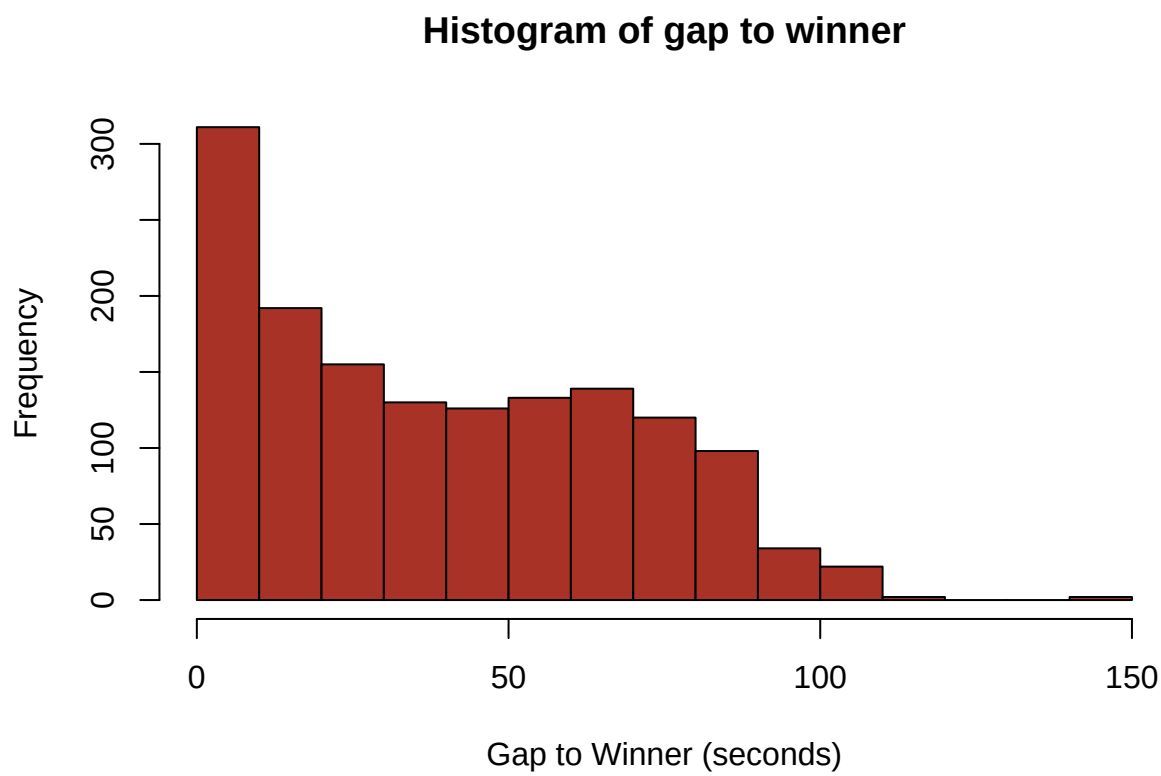
```r
full_data %>%
  select(grid, gap_seconds) %>%
  summary()
```

```
##       grid          gap_seconds
##  Min.   : 1.000   Min.   :  0.00
##  1st Qu.: 4.000   1st Qu.: 12.98
##  Median : 7.000   Median : 35.59
##  Mean   : 8.416   Mean   : 39.26
##  3rd Qu.:13.000   3rd Qu.: 62.95
##  Max.   :20.000   Max.   :148.84
```

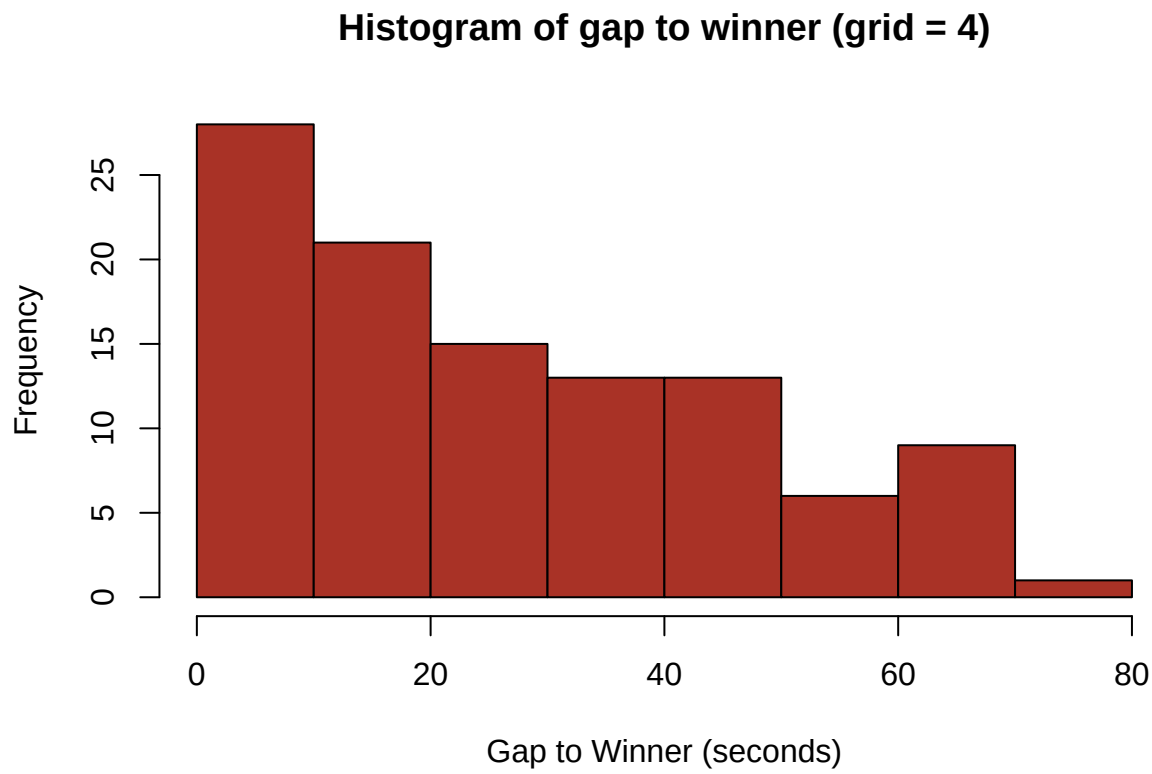
Grid positions in the cleaned dataset range from 1 to 20 with a mean of 8.4, while gap to winner ranges from

0 to 148.8 seconds with a mean of 39.3 seconds. The large difference between the mean and the maximum gap, along with the right-skewed histogram, suggests that a normal likelihood would be inappropriate for modelling this response variable.

```
hist(full_data$gap_seconds,
     col = "#a93226",
     main = "Histogram of gap to winner",
     xlab = "Gap to Winner (seconds)")
```



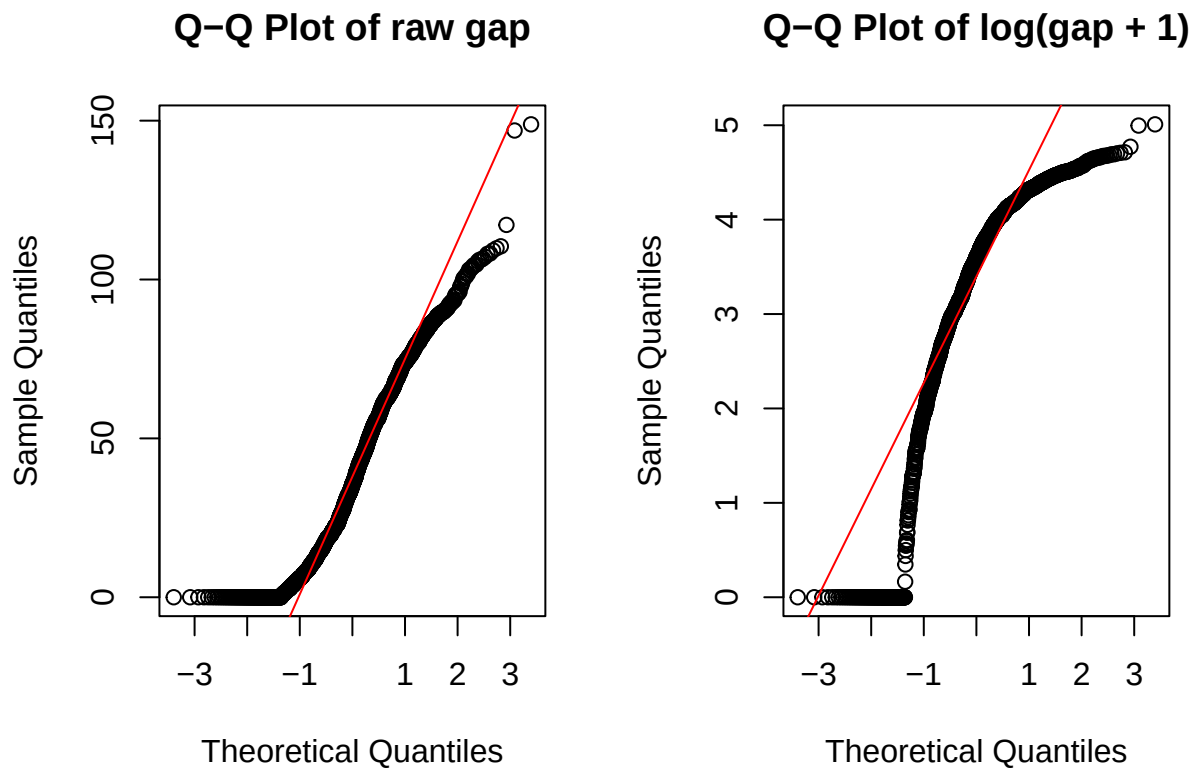

```
hist(filter(full_data, grid == 4)$gap_seconds,
     col = "#a93226",
     main = "Histogram of gap to winner (grid = 4)",
     xlab = "Gap to Winner (seconds)")
```



The distribution of gap to winner is heavily right-skewed, with most observations concentrated below 50 seconds and a long tail extending toward 150 seconds. The response is also strictly non-negative, which rules out the use of a Normal likelihood and motivates the choice of a distribution with positive support such as the Exponential.

Even when conditioning on a single grid position, the gap distribution remains right-skewed and strictly positive. This suggests the skewness is not simply a consequence of mixing different grid positions and is a genuine feature of the data.

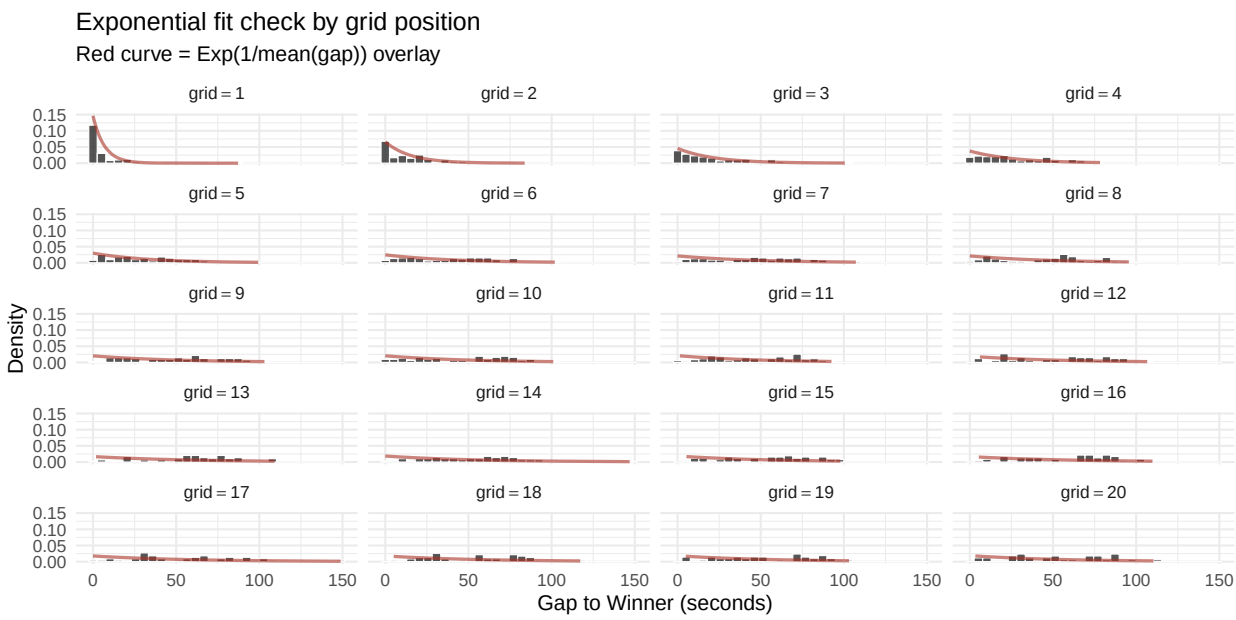
```
par(mfrow = c(1, 2))

qqnorm(full_data$gap_seconds,
       main = "Q-Q Plot of raw gap")
qqline(full_data$gap_seconds, col = "red")

qqnorm(log(full_data$gap_seconds + 1),
       main = "Q-Q Plot of log(gap + 1)")
qqline(log(full_data$gap_seconds + 1), col = "red")
```



Both the raw gap and log-transformed gap deviate substantially from the normal reference line. The raw gap shows severe right skew, and while the log transformation reduces this, the departure from normality remains visible particularly in the lower tail. These results suggest that neither a Normal nor a log-Normal likelihood is well-suited to this data, further supporting the choice of an Exponential likelihood.

```
#Exponential fit check by grid position and rate = 1/mean(gap)

full_data |>
  group_by(grid) |>
  mutate(lambda = 1 / mean(gap_seconds)) |>
  ungroup() |>
  ggplot(aes(x = gap_seconds)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 30,
                 fill = "#2b2b2b",
                 alpha = 0.8,
                 colour    = "white",
                 linewidth = 0.3) +
  geom_line(aes(x = gap_seconds, y = dexp(gap_seconds, rate = lambda)),
            color = "#a93226",
            alpha = 0.6,
            linewidth = 0.8) +
  facet_wrap(~ grid, ncol = 4,
             labeller = label_bquote(grid == .(grid))) +
  labs(
    title = "Exponential fit check by grid position",
    subtitle = "Red curve = Exp(1/mean(gap)) overlay",
```

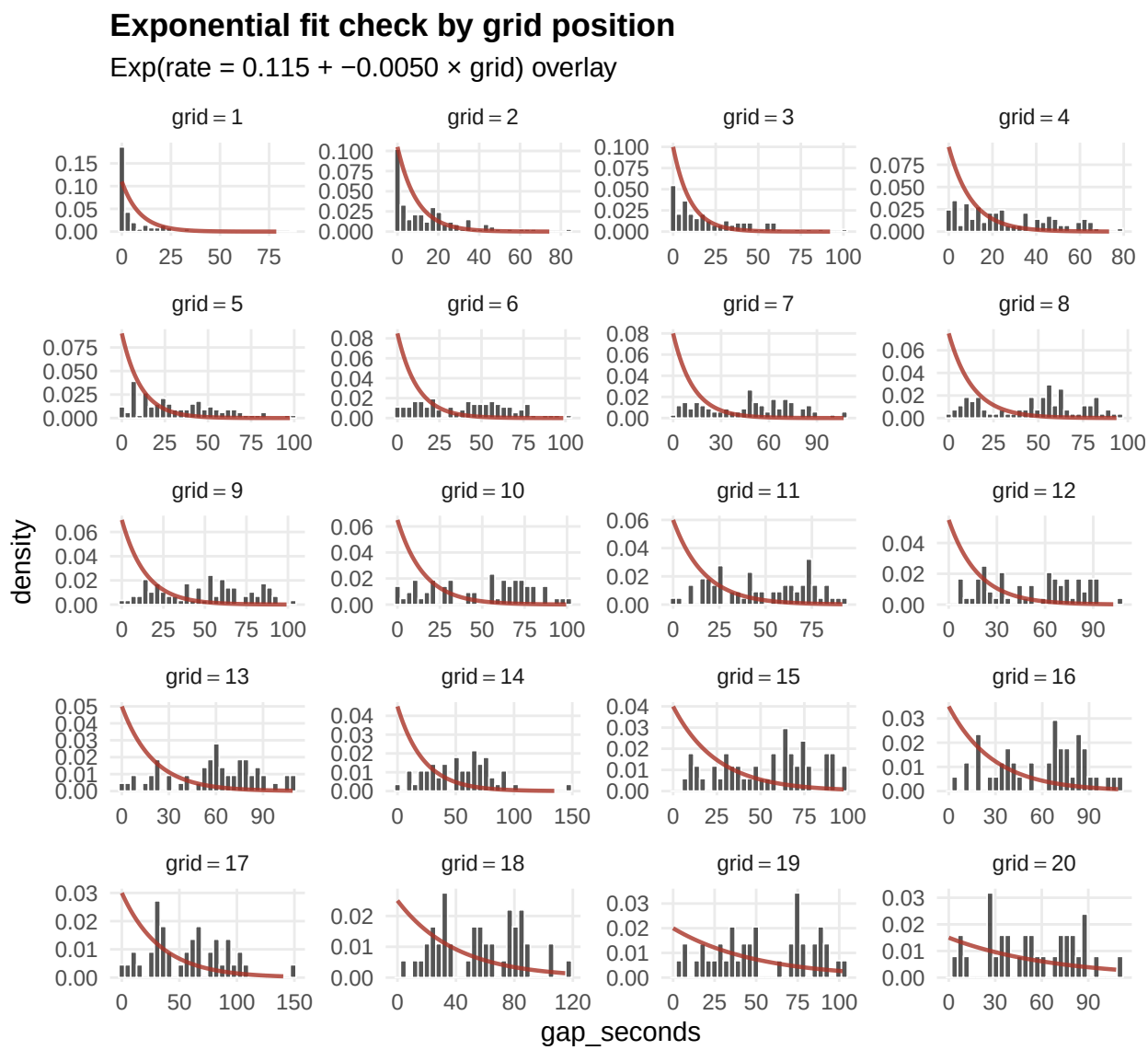
```r
    x = "Gap to Winner (seconds)",
    y = "Density"
  ) +
  theme_minimal()
```



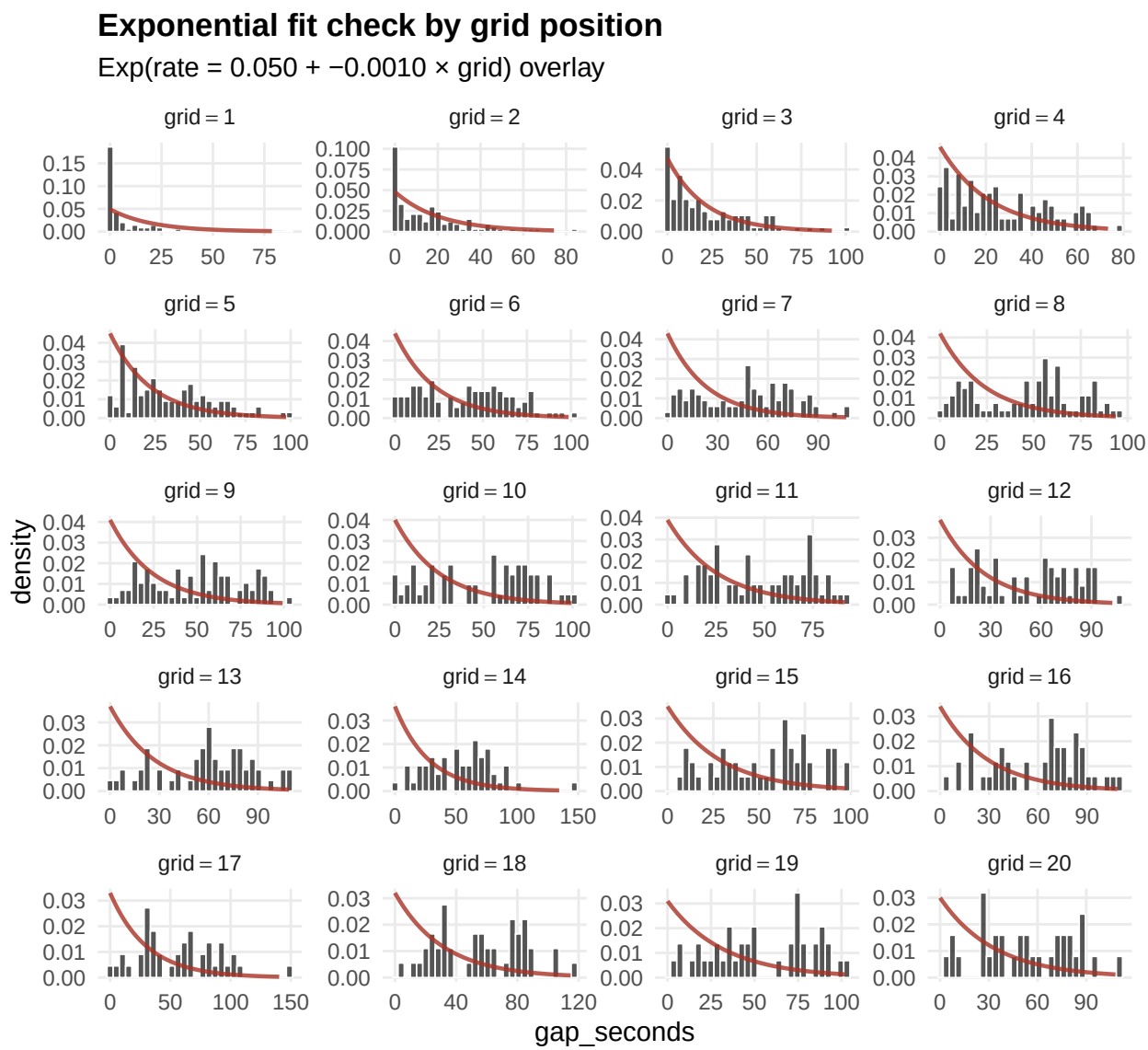
Using a lambda = 1/mean(gap) for each grid position separately, the Exponential distribution captures the overall shape of the data reasonably well, particularly at lower grid positions where the fit is closest. The fit is less precise at higher grid positions, which is expected given the mean-variance constraint of the Exponential distribution. Overall, this provides reasonable support for the Exponential likelihood as a working approximation.

```r
# Exponential overlay function to test different alpha/beta values
# To understand reasonable priors
plot_exp_overlay <- function(alpha, beta) {
  params <- tibble(grid = 1:20) |>
    mutate(rate = alpha + beta * grid)

  overlay <- params |>
    rowwise() |>
    mutate(
      x    = list(seq(0, quantile(full_data$gap_seconds[full_data$grid == grid], 0.995),
                      length.out = 300)),
      dens = list(dexp(unlist(x), rate = rate))
    ) |>
    unnest(c(x, dens))

  ggplot(full_data, aes(x = gap_seconds)) +
    geom_histogram(
      aes(y = after_stat(density)),
      bins      = 30,
      fill      = "#2b2b2b",
      alpha     = 0.8,
      colour    = "white",
      linewidth = 0.3
```

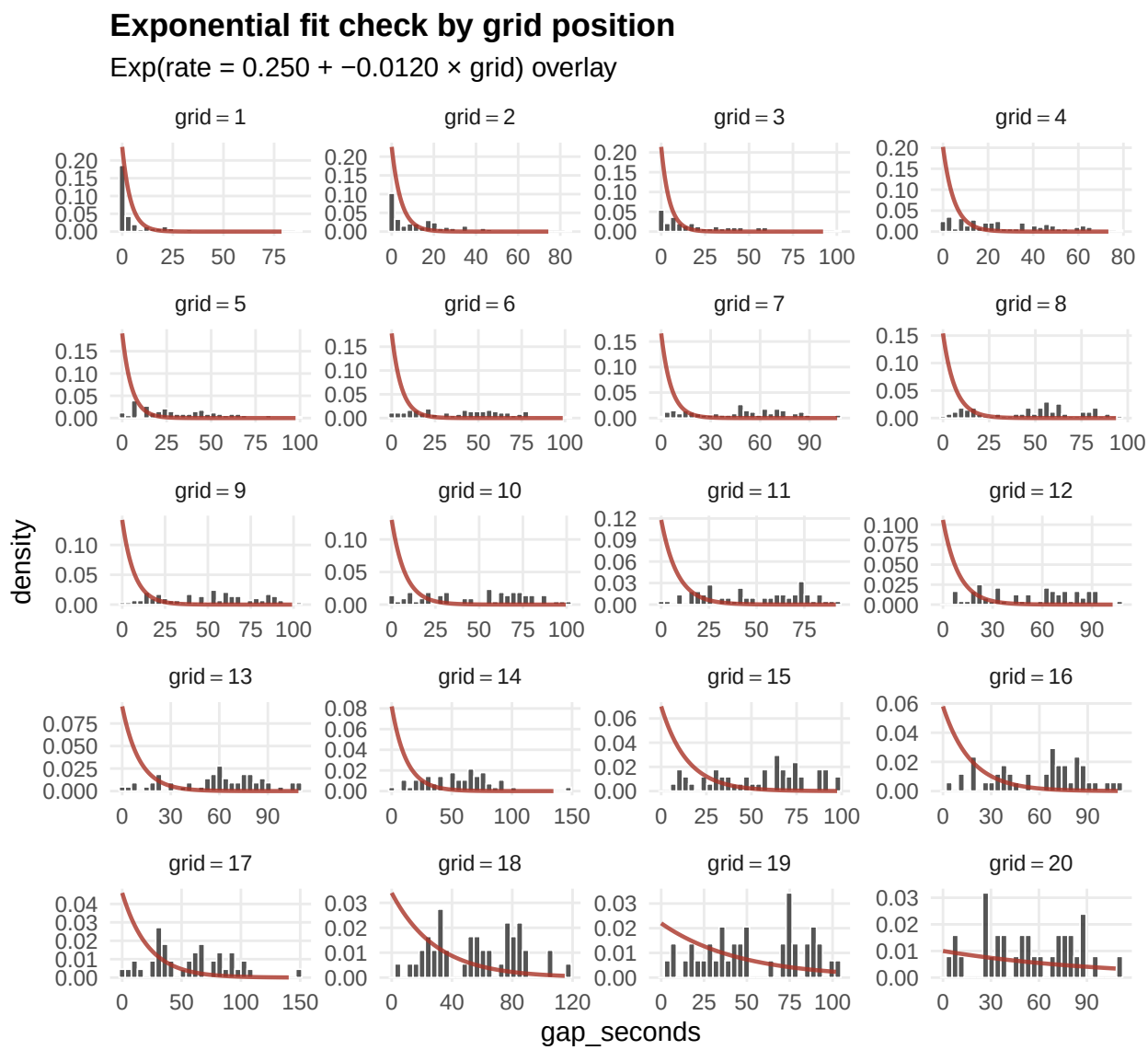
```r
    ) +
    geom_line(
      data      = overlay,
      aes(x = x, y = dens),
      colour    = "#a93226",
      alpha     = 0.8,
      linewidth = 0.8
    ) +
    facet_wrap(~ grid, scales = "free", ncol = 4,
               labeller = label_bquote(grid == .(grid))) +
    labs(
      x        = "gap_seconds",
      y        = "density",
      title    = "Exponential fit check by grid position",
      subtitle = sprintf("Exp(rate = %.3f + %.4f × grid) overlay", alpha, beta)
    ) +
    theme_minimal(base_size = 11) +
    theme(
      strip.text       = element_text(size = 9, face = "bold"),
      panel.grid.minor = element_blank(),
      plot.title       = element_text(face = "bold")
    )
}

# Test 3 different ranges
plot_exp_overlay(alpha = 0.115, beta = -0.005)
```



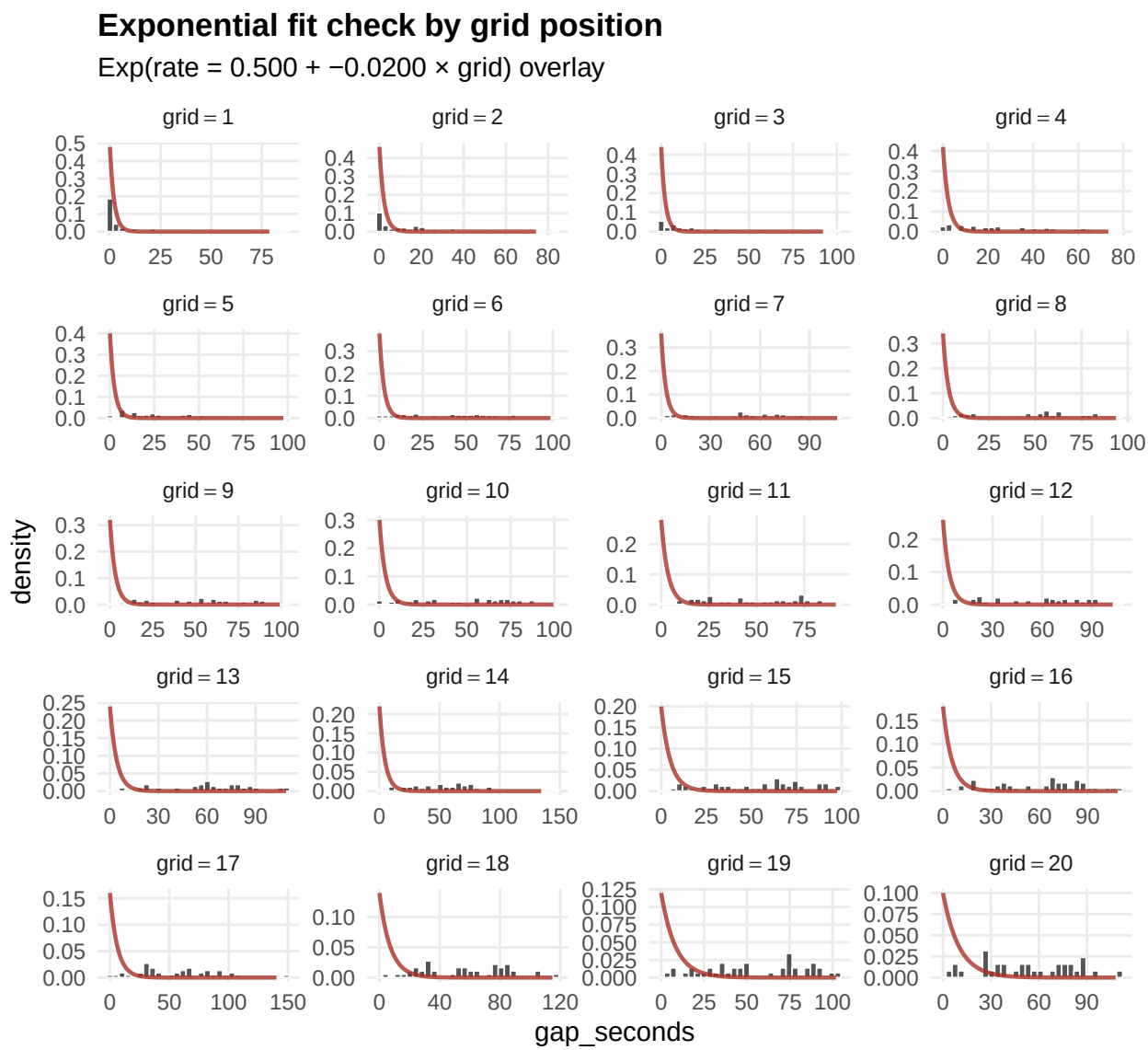

```
plot_exp_overlay(alpha = 0.05,  beta = -0.001)
```




```
plot_exp_overlay(alpha = 0.25,  beta = -0.012)
```



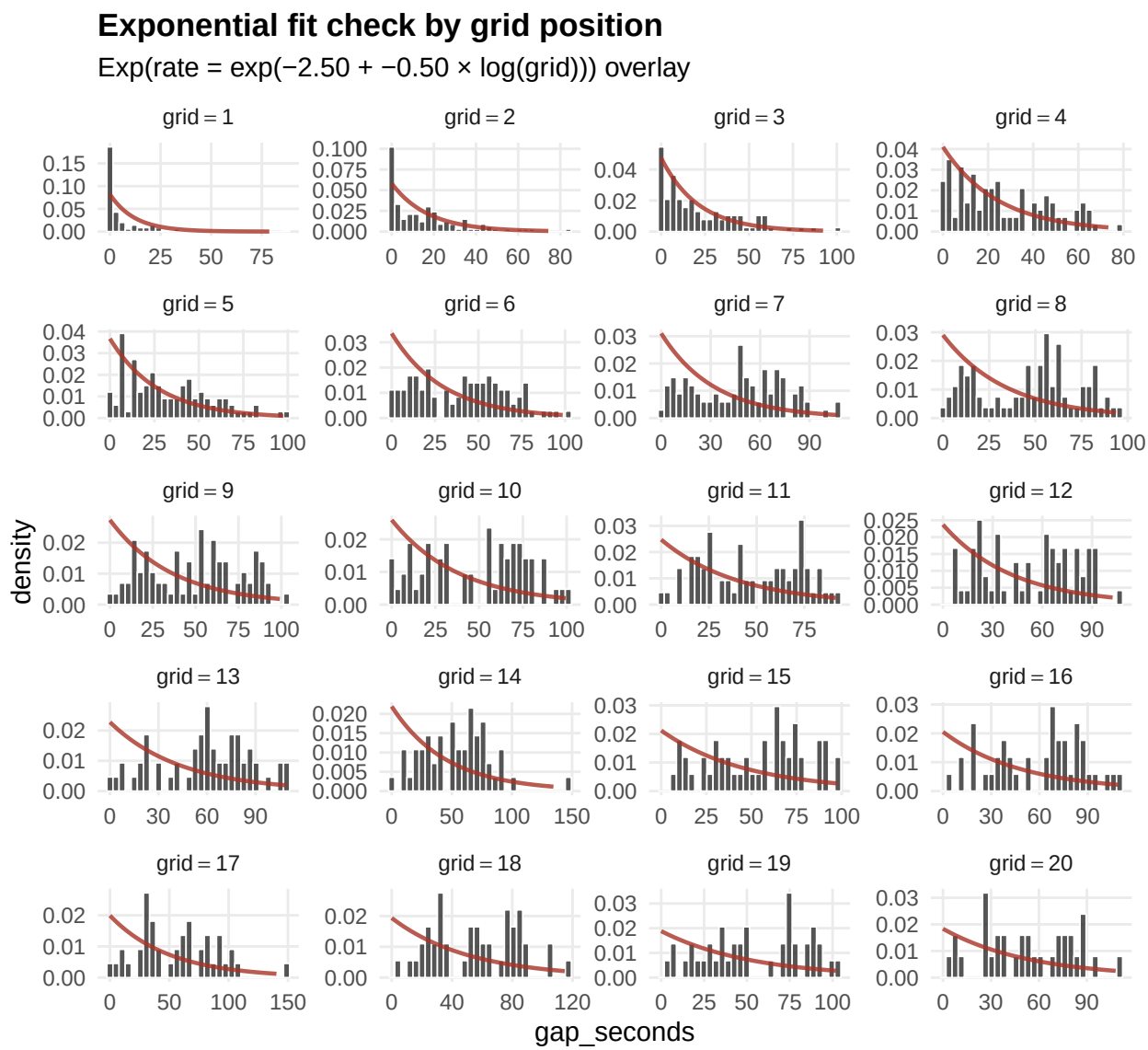

```
plot_exp_overlay(alpha = 0.50,  beta = -0.020)
```



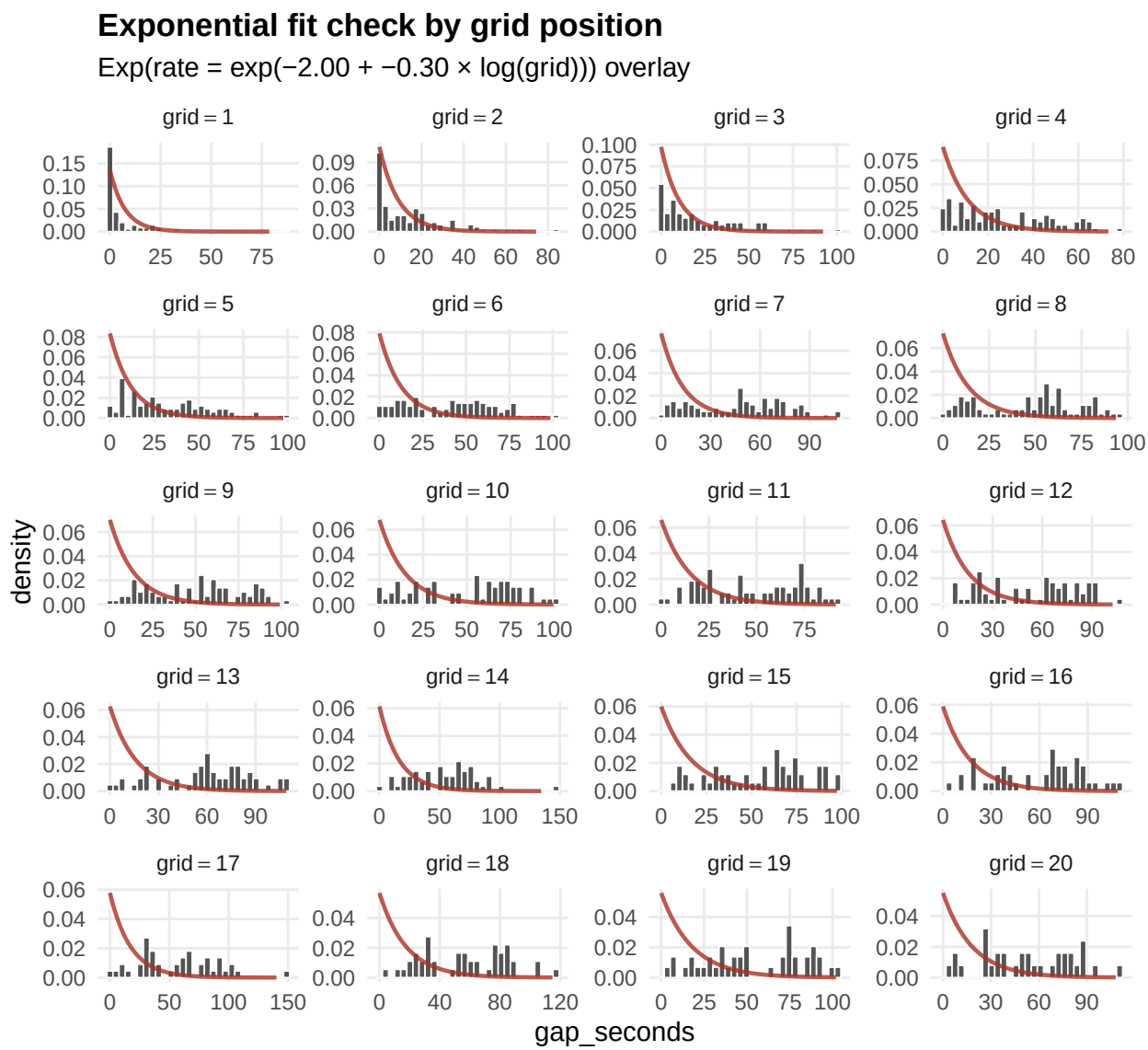
Three different values of alpha and beta are tested using a linear rate parameterization. In all cases the exponential curve fails to track the histogram well across all grid positions, as the linear predictor cannot adequately capture the concave relationship seen in the nonlinearity check. This reinforces the motivation for using a log(grid) transformation of the predictor.

```
plot_exp_overlay <- function(alpha, beta) {
  params <- tibble(grid = 1:20) |>
    mutate(rate = exp(alpha + beta * log(grid)))

  overlay <- params |>
    rowwise() |>
    mutate(
      x    = list(seq(0, quantile(full_data$gap_seconds[full_data$grid == grid], 0.995),
                      length.out = 300)),
      dens = list(dexp(unlist(x), rate = rate))
    ) |>
    unnest(c(x, dens))
```

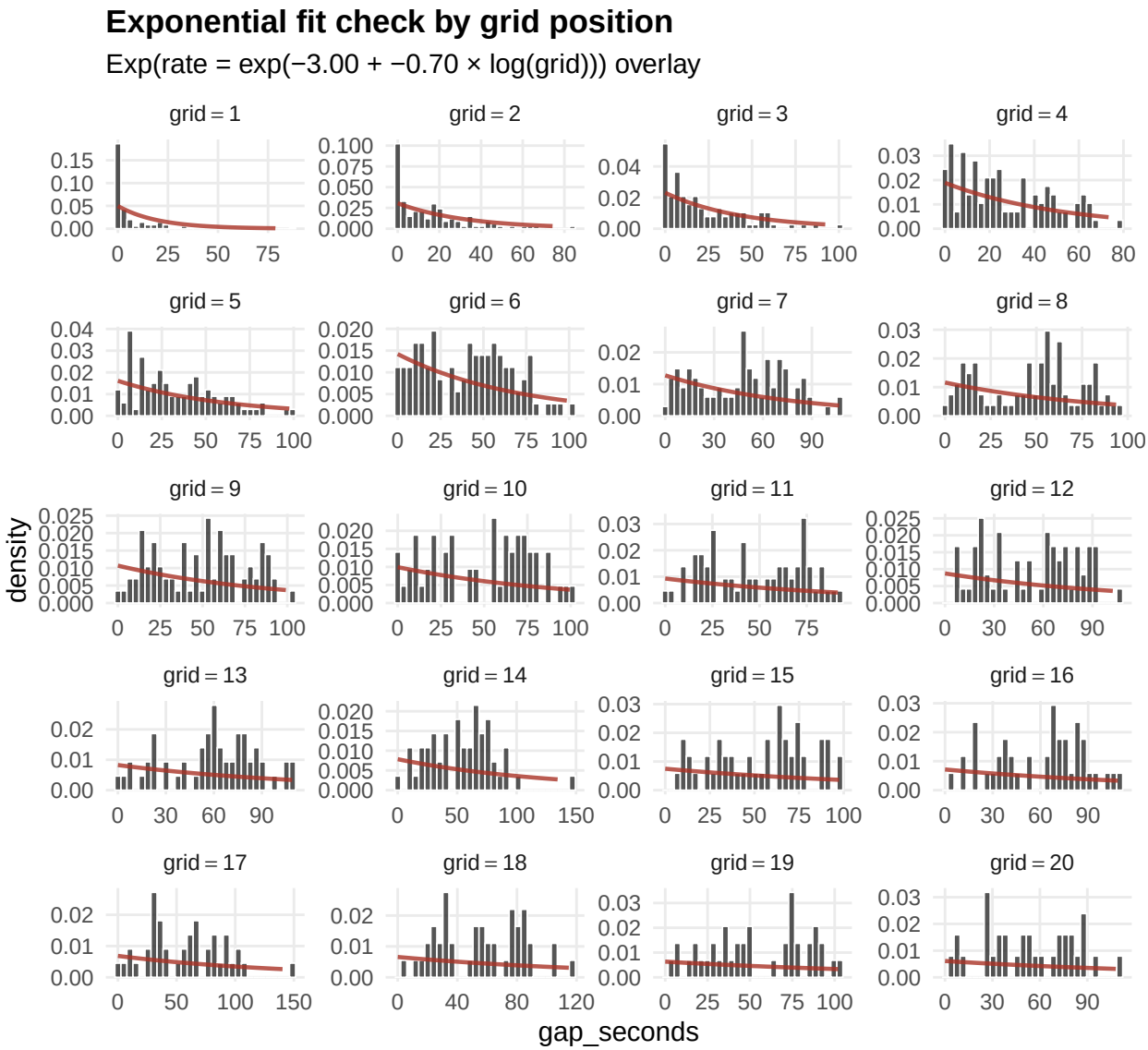
```r
  ggplot(full_data, aes(x = gap_seconds)) +
    geom_histogram(
      aes(y = after_stat(density)),
      bins      = 30,
      fill      = "#2b2b2b",
      alpha     = 0.8,
      colour    = "white",
      linewidth = 0.3
    ) +
    geom_line(
      data      = overlay,
      aes(x = x, y = dens),
      colour    = "#a93226",
      alpha     = 0.8,
      linewidth = 0.8
    ) +
    facet_wrap(~ grid, scales = "free", ncol = 4,
               labeller = label_bquote(grid == .(grid))) +
    labs(
      x        = "gap_seconds",
      y        = "density",
      title    = "Exponential fit check by grid position",
      subtitle = sprintf("Exp(rate = exp(%.2f + %.2f × log(grid))) overlay", alpha, beta)
    ) +
    theme_minimal(base_size = 11) +
    theme(
      strip.text       = element_text(size = 9, face = "bold"),
      panel.grid.minor = element_blank(),
      plot.title       = element_text(face = "bold")
    )
}

# Test 3 different ranges
plot_exp_overlay(alpha = -2.5, beta = -0.5)
```




```
plot_exp_overlay(alpha = -2.0, beta = -0.3)
```




```r
plot_exp_overlay(alpha = -3.0, beta = -0.7)
```



Replacing the linear predictor with log(grid) produces a better visual fit across grid positions compared to the linear version. Among the values tested, alpha around -2.5 and beta around -0.5 appear to provide the most reasonable coverage of the observed data, and these values will inform our prior specification in the Stan model.